

Métodos de Manipulação do DOM

Nesta leitura, você aprenderá sobre um método de manipulação do Modelo de Objeto de Documento (DOM) conhecido como `querySelectorAll`.

`querySelectorAll`

`querySelectorAll` é um método em JavaScript que seleciona múltiplos elementos HTML dentro do DOM com base em seletores semelhantes ao CSS. Ele retorna uma coleção (um `NodeList` não ao vivo) de elementos que correspondem ao seletor especificado. Você pode usá-lo para selecionar elementos por classe, ID ou nome de tag.

Aqui estão exemplos de como usar `querySelectorAll` para seleções de classe, ID e tag com `console.log` e explicações de sua sintaxe:

1. Selecionando por Classe:

Código HTML

```
<html>
<head>
  <title>Exemplo de querySelectorAll</title>
</head>
<body>
  <p class="highlighted">Este é um parágrafo destacado.</p>
  <p class="highlighted">Este é outro parágrafo destacado.</p>
  <p>Este é um parágrafo normal.</p>
</body>
</html>
```

```
JavaScript Code
````// Select all elements with the class "highlighted" using querySelectorAll
const elementsByClass = document.querySelectorAll('.highlighted');
// Log the selected elements to the console
console.log(elementsByClass);
```

Saída

```
NodeList [<p.highlighted>, <p.highlighted>]
```

### Explicação:

- `document.querySelectorAll('.highlighted')` seleciona todos os elementos com a classe "highlighted" dentro do documento.
- A coleção `elementsByClass` armazena os elementos selecionados, que formam uma `NodeList`.
- `console.log(elementsByClass);` registra os elementos selecionados no console.
- Explicação da Saída: A `NodeList elementsByClass` contém dois `<p>` elementos com a classe "highlighted." A declaração `console.log` exibe a `NodeList` com esses dois elementos.

### 2. Selecionando por ID:

Código HTML

```
<!DOCTYPE html>
<html>
<head>
 <title>querySelectorAll Example</title>
</head>
<body>
 <p id="my-paragraph">This is a paragraph with an ID.</p>
 <p>This is another paragraph.</p>
</body>
</html>
```

#### Código JavaScript

```
// Select the element with the ID "my-paragraph" using querySelectorAll
const elementByID = document.querySelectorAll('#my-paragraph');
// Log the selected element to the console
console.log(elementByID);
```

#### Saída

```
NodeList [<p#my-paragraph>]
```

#### Explicação:

- `document.querySelectorAll('#my-paragraph')` seleciona o elemento com o ID "my-paragraph" dentro do documento. Mesmo que `querySelectorAll` seja usado, ele ainda retorna uma coleção, mas neste caso, contém apenas um elemento (se o ID for único).
- A coleção `elementByID` armazena os elementos selecionados, que formam um `NodeList`.
- `console.log(elementByID)`; registra o elemento selecionado no console.
- Explicação da Saída: O `NodeList elementByID` contém o `<p>` elemento com o ID "my-paragraph." Mesmo sendo um único elemento, ele ainda é representado como um `NodeList`. A declaração `console.log` exibe o `NodeList` com este elemento.

### 3. Selecionando pelo Nome da Tag:

#### Código HTML

```
<!DOCTYPE html>
<html>
<head>
 <title>querySelectorAll Example</title>
</head>
<body>
 <p>This is a paragraph.</p>
 <p>This is another paragraph.</p>
 <p class="highlighted">This is a highlighted paragraph.</p>
</body>
</html>
```

#### Código JavaScript

```
// Select all <p> elements using querySelectorAll
const elementsByTag = document.querySelectorAll('p');
// Log the selected elements to the console
console.log(elementsByTag);
```

Output

```
▼ NodeList(3) [p, p, p.highlighted] ⓘ
 ► 0: p
 ► 1: p
 ► 2: p.highlighted
 length: 3
 ► [[Prototype]]: NodeList
```

#### Explicação:

- `document.querySelectorAll('p')` seleciona todos os elementos `<p>` dentro do documento.
- Os elementos selecionados são armazenados na coleção `elementsByTag`, que é uma `NodeList`.
- `console.log(elementsByTag)`; registra os elementos selecionados no console.
- Explicação da Saída: A `NodeList elementsByTag` contém todos os três elementos `<p>` no documento. A instrução `console.log` exibe a `NodeList` com esses três elementos.

## classList

A propriedade `classList` é um recurso útil que permite manipular classes em elementos HTML de forma fácil. Vamos mergulhar em uma visão geral da propriedade `classList` e seus métodos.

### A Propriedade `classList` em JavaScript

No DOM, a propriedade `classList` está associada a um elemento HTML e fornece uma coleção de métodos para trabalhar com as classes do elemento.

#### Acessando `classList`

Você pode acessar a propriedade `classList` de um elemento usando JavaScript assim:

```
const element = document.getElementById('myElement');
const classes = element.classList;
```

#### Métodos Comuns de `classList`

##### 1. `add(class1, class2, ...)`

Este método adiciona uma ou mais classes ao elemento.

```
element.classList.add('newClass');
```

##### 2. `remove(class1, class2, ...)`

Remove uma ou mais classes do elemento.

```
element.classList.remove('oldClass');
```

##### 3. `toggle(class, force)`

Alterna uma classe. Se a classe existir, ela é removida; caso contrário, é adicionada. Se o segundo parâmetro for verdadeiro, a classe é adicionada; se falso, a classe é removida.

```
element.classList.toggle('active');
```

#### 4. contains(class)

Verifica se uma classe está presente no elemento. Retorna verdadeiro se a classe existir; caso contrário, é falso.

```
if (element.classList.contains('special')) {
 // Do something special
}
```

#### 5. replace(oldClass, newClass)

Substitui uma classe por outra classe.

```
element.classList.replace('oldClass', 'newClass');
```

#### 6. item(index)

Retorna o nome da classe no índice especificado.

```
const firstClass = element.classList.item(0);
```

#### 7. toString()

Retorna uma string que representa as classes do elemento.

```
const classString = element.classList.toString();
```

## Exemplo

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>classList Example</title>
 <style>
 .highlight {
 color: red;
 font-weight: bold;
 }
 .italic {
 font-style: italic;
 }
 .underline {
 text-decoration: underline;
 }
 .strike {
```

```

 text-decoration: line-through;
 }
</style>
</head>
<body>

 <p id="myParagraph" class="highlight">This is a paragraph.</p>
 <button onclick="performClassListOperations()">Perform Operations</button>

 <script>
 function performClassListOperations() {
 const paragraph = document.getElementById('myParagraph');

 // Adding a class
 paragraph.classList.add('italic');

 // Removing a class
 paragraph.classList.remove('highlight');

 // Toggling a class
 paragraph.classList.toggle('underline', true);

 // Checking if a class exists
 const hasItalicClass = paragraph.classList.contains('italic');
 console.log(`Has italic class: ${hasItalicClass}`);

 // Replacing a class after a delay (for demonstration)
 setTimeout(() => {
 paragraph.classList.replace('underline', 'strike');

 // Accessing classes as a string
 const classString = paragraph.classList.toString();
 console.log(`Current classes: ${classString}`);
 }, 2000); // Delay for 2 seconds
 }
 </script>

</body>
</html>

```

Vamos detalhar passo a passo:

### Estrutura HTML

- `<!DOCTYPE html>` Especifica a versão do HTML que está sendo usada.
- `<html lang="en">` Declara o idioma do documento como inglês.
- A seção `<head>` contém metadados como codificação de caracteres, configurações de viewport e o título do documento.
- Dentro do `<head>`, há um bloco `<style>` definindo a tag de estilo para aplicar CSS interno.

### Conteúdo do Corpo

- `<p id="myParagraph" class="highlight">Este é um parágrafo.</p>` Este elemento de parágrafo HTML (`<p>`) tem um ID de “myParagraph” e uma classe de “highlight.” É o elemento sobre o qual realizaremos operações de `classList`.
- `<button onclick="performClassListOperations()">Realizar Operações</button>` Este botão, ao ser clicado, aciona a função `performClassListOperations()`.

### Seção JavaScript

Função JavaScript `performClassListOperations()`

Esta função é executada quando um botão, provavelmente chamado “Executar Operações”, é clicado no HTML. Aqui está uma explicação detalhada de cada etapa dentro da função:

#### 1. Obtendo o Elemento de Parágrafo

```
const paragraph = document.getElementById('myParagraph');
```

- `const paragraph:` Declara uma variável chamada `paragraph`.
- `document.getElementById('myParagraph')` Recupera o elemento HTML com o ID "myParagraph" e o atribui à variável `paragraph`.

#### 2. Adicionando uma Classe

```
paragraph.classList.add('italic');
```

- `paragraph.classList.add('italic')` Adiciona a classe "italic" à lista de classes do elemento de parágrafo.

### 3. Removendo uma Classe

```
paragraph.classList.remove('highlight');
```

### 4. Alternando uma Classe

```
paragraph.classList.toggle('underline', true);
```

- `paragraph.classList.toggle('underline', true)` Alterna a classe "underline" no elemento de parágrafo. Neste caso, ela adiciona explicitamente a classe "underline" porque o segundo parâmetro é verdadeiro.

### 5. Verificando se uma Classe Existe

```
const hasItalicClass = paragraph.classList.contains('italic');
console.log(`Has italic class: ${hasItalicClass}`);
```

- `paragraph.classList.contains('italic')` Verifica se a classe "italic" existe na lista de classes do elemento parágrafo.
- O resultado (verdadeiro ou falso) é armazenado na variável `hasItalicClass` e registrado no console.

### 6. Substituindo uma Classe com um Atraso

```
setTimeout(() => {
 paragraph.classList.replace('underline', 'strike');

 // Accessing classes as a string
 const classString = paragraph.classList.toString();
 console.log(`Current classes: ${classString}`);
}, 2000); // Delay for 2 seconds
```

- `setTimeout(() => { ... }, 2000)` Retarda a execução do código interno em 2000 milissegundos (2 segundos).
- Dentro da função de timeout:
  - `paragraph.classList.replace('underline', 'strike')` Substitui a classe "underline" por "strike" na lista de classes do elemento parágrafo.
  - `const classString = paragraph.classList.toString()` Recupera as classes atualizadas como uma string.
  - `console.log(Current classes: ${classString})` Registra as classes atuais do elemento parágrafo no console.

## Resumo

Esta leitura demonstra como manipular dinamicamente as classes de um elemento HTML usando a propriedade `classList` do JavaScript. Ela mostra como adicionar, remover, alternar, verificar a existência e substituir classes, oferecendo um exemplo prático de manipulação de classes dentro de uma página da web.



# Skills Network

## Changelog

Data	Versão	Alterado por	Descrição da Mudança
2023-12-14	0.2	Pooja Bhardwaj	Conteúdo atualizado

© IBM Corporation 2023. Todos os direitos reservados.